Module 6: Critical Thinking
Option 2: Distinguishing Dogs & Cats

Christine DeLuna
CSC 580: Dr. Nguyen
August 26, 2026

Introduction

Convolutional neural networks (CNNs) are a form of deep learning designed to identify spatial patterns within data, making them particularly useful for image classification and computer vision. Unlike a traditional fully connected neural network, which can flatten an image into a single vector of pixel values, a CNN applies learned filters across an image to detect spatial features such as edges, shapes, textures, and increasingly complex visual patterns. This ability to preserve and learn from spatial relationships makes CNNs well suited for distinguishing visually different classes, including the cats and dogs examined in this project.

The purpose of this project was to develop and evaluate neural-network models for binary image classification using a dataset containing images of cats and dogs. A fully connected neural network was first implemented as a baseline and compared with a CNN containing a convolutional layer, max pooling, and two fully connected layers. The CNN architecture was then expanded with a second convolutional layer containing 48 filters to determine whether additional feature extraction improved classification performance. Model performance was evaluated using validation accuracy, mean squared error, Pearson correlation, predicted class probabilities, and individual image classifications.

Beyond determining whether the models could distinguish cats from dogs, this experiment provides an opportunity to examine a broader issue in artificial intelligence: increasing model complexity does not necessarily result in a proportionate improvement in performance or reliability. The results demonstrate that convolutional architectures can improve upon a simple fully connected approach for image data, but they also reveal substantial classification error, variability, and instances in which the CNN produces highly confident incorrect predictions. Consequently, model quality must be evaluated using performance across unseen data rather than architecture complexity or confidence scores alone. This distinction is particularly important when considering the use of increasingly sophisticated artificial intelligence models in real-world decision-making.

Dataset Acquisition and Preparation

The original assignment specified the use of Kaggle's *Dogs vs. Cats* competition dataset. During implementation, however, access to the competition files presented an unexpected technical limitation. Although the competition rules had been accepted and a Kaggle API token was successfully configured, attempts to retrieve the dataset programmatically returned a 403 Forbidden response. An alternative TensorFlow-hosted download of the filtered cats-and-dogs dataset also returned a 403 response. Rather than allowing these external access restrictions to prevent completion of the experiment, TensorFlow Datasets (TFDS) was used as an alternative method for obtaining the cats-versus-dogs image data. This also illustrates a practical consideration in machine learning development: reproducibility depends not only on source code, but also on the continued availability and accessibility of external datasets.

The resulting TFDS dataset contained 23,262 images, including 11,658 cat images and 11,604 dog images. The images varied in their original dimensions, so they were exported into separate local `cats` and `dogs` directories and processed according to the resizing approach specified in

the assignment. Each image was resized to 94 × 125 pixels and represented using three RGB color channels. Verification of the preprocessing procedure showed a resulting NumPy image shape of (125, 94, 3) for all 1,000 images initially inspected. Pixel values remained in their original 0–255 range.

For model development, 2,048 images from each class were selected for training, producing a balanced training set of 4,096 images. An additional 512 cat and 512 dog images were reserved for validation, resulting in 1,024 validation images. The final training array therefore had a shape of (4096, 125, 94, 3), while the validation array had a shape of (1024, 125, 94, 3). Cats were assigned a binary label of 1 and dogs a label of 0. Because the cat and dog images were initially concatenated by class, shuffling during model training was particularly important to prevent the network from receiving all examples of one class before examples of the other.

Figure 1 documents verification of the resized image dimensions, while Figure 2 summarizes the final training and validation datasets. Together, these preprocessing steps created a consistent and balanced dataset that could be used to compare the fully connected baseline model with the subsequent convolutional architectures.

Figure 1
Image Resizing and Shape Verification for the Cats vs. Dogs Dataset

```
/Users/christine/.pyenv/versions/3.11.6/bin/python3 /Users/christine/PycharmProjects/CSC580/assignments/CTA6-CNN-Cats-Dogs/cnn_cats_dogs.py
============================================================
CSC580 - CNN Cats vs. Dogs Classification
============================================================

Checking resized image dimensions...
Processed 0 images
Processed 100 images
Processed 200 images
Processed 300 images
Processed 400 images
Processed 500 images
Processed 600 images
Processed 700 images
Processed 800 images
Processed 900 images

Most common resized image shapes:
(125, 94, 3): 1000

Sample image shape: (125, 94, 3)

Step 1A complete.
============================================================

Process finished with exit code 0
```

Note. The output verifies the preprocessing procedure used to prepare image data for neural network training. A sample of 1,000 cat images was resized using the assignment-specified dimensions of 94 × 125 pixels. Because PIL represents image size as width × height while NumPy represents arrays as height × width × channels, each resulting image has a shape of (125, 94, 3). All 1,000 sampled images produced the same dimensions, with three channels representing RGB color values.

Figure 2
Training and Validation Dataset Configuration for CNN Classification

```
============================================================
Loading Training and Validation Data
============================================================

Loading training cat images...
Loading training dog images...
Loading validation cat images...
Loading validation dog images...

Dataset Summary
------------------------------------------------------------
Training image shape:    (4096, 125, 94, 3)
Training label shape:    (4096,)
Validation image shape:  (1024, 125, 94, 3)
Validation label shape:  (1024,)

Training cats:  2048
Training dogs:  2048
Validation cats: 512
Validation dogs: 512

Pixel value range:
Minimum: 0
Maximum: 255

Step 1B complete.
============================================================

Process finished with exit code 0
```

Note. The prepared dataset was divided into balanced training and validation samples for binary image classification. The training set contained 4,096 images, consisting of 2,048 cat images and 2,048 dog images, while the validation set contained 1,024 images, consisting of 512 images from each class. All images were represented as 125 × 94 × 3 NumPy arrays corresponding to image height, width, and RGB color channels. Pixel values remained in their original range of 0 to 255 to preserve the preprocessing approach specified in the assignment.

Baseline Fully Connected Neural Network

A fully connected neural network was first developed to establish a baseline against which the convolutional models could be compared. Following the architecture provided in the assignment, each 125 × 94 RGB image was flattened from its three-dimensional representation into a vector containing 35,250 individual pixel values. This vector was passed to a dense hidden layer containing 512 neurons with ReLU activation, followed by a single sigmoid output neuron representing the predicted probability that an image belonged to the cat class. Although relatively simple in structure, flattening the images resulted in a model containing 18,049,025 trainable parameters, with the majority occurring between the flattened input and first dense layer.

The baseline model was trained using the Adam optimizer with a learning rate of 0.001, mean squared error (MSE) as the loss function, a batch size of 32, and 10 training epochs. The training data were shuffled during each epoch because the image arrays had initially been constructed by placing the cat images before the dog images. Binary cross-entropy and MSE were monitored

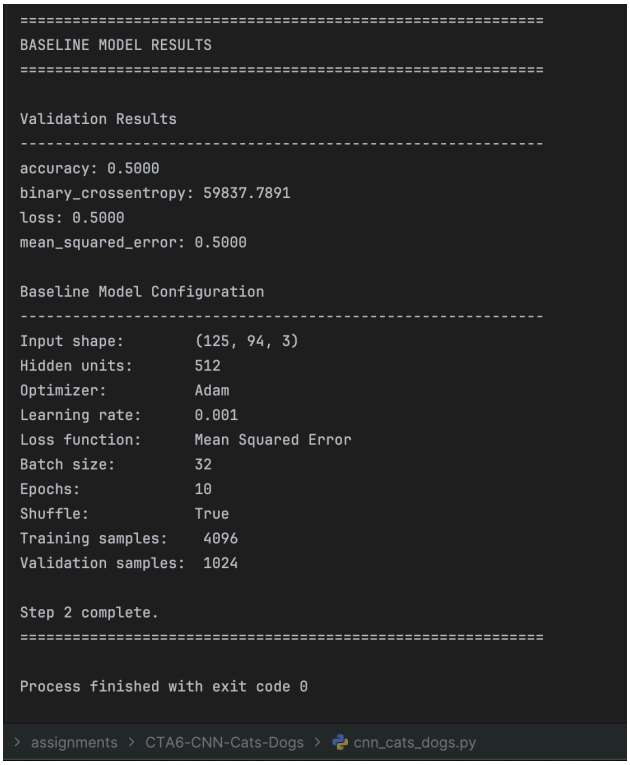
during training, and accuracy was included as an additional metric to provide a more intuitive assessment of binary classification performance.

Despite its large number of trainable parameters, the baseline network achieved a validation accuracy of only 50.00%, with a validation MSE of 0.5000. Because the validation dataset contained equal numbers of cats and dogs, an accuracy of 50% is equivalent to chance-level classification. The model therefore failed to learn a useful distinction between the two image classes. The extremely high reported binary cross-entropy value further indicated that the model was producing poor probability estimates, even though MSE was the optimization objective.

These results demonstrate an important limitation of applying a standard fully connected architecture directly to image data. Flattening converts the image into a long sequence of pixel values and removes the explicit two-dimensional relationships among neighboring pixels. The network can therefore no longer directly exploit spatial features such as edges, textures, shapes, and their relative locations within an image. Furthermore, the model required more than 18 million trainable parameters while still producing chance-level validation performance. In this case, a large parameter count did not translate into effective learning or generalization.

Figure 3 presents the baseline model results and configuration. The poor performance of this architecture provides a useful comparison for evaluating whether convolutional layers, which are specifically designed to learn spatial image features, provide a meaningful advantage for the cats-versus-dogs classification task.

Figure 3
Validation Performance of the Fully Connected Baseline Model



Note. The fully connected baseline model achieved a validation accuracy of 50.00% after 10 epochs using the Adam optimizer and mean squared error loss. Because the validation dataset contained equal numbers of cats and dogs, this result represents approximately chance-level classification performance. The model flattened each 125 × 94 × 3 image into 35,250 individual pixel values before processing them through a 512-unit dense layer, resulting in more than 18 million trainable parameters. The poor validation performance provides a baseline for evaluating whether convolutional layers can more effectively learn spatial features from the same image data.

Convolutional Neural Network Model

Following evaluation of the fully connected baseline, a convolutional neural network was implemented to determine whether preserving and learning from spatial relationships within the images would improve classification performance. Unlike the baseline model, which immediately flattened each image into 35,250 independent pixel values, the CNN first applied a two-dimensional convolutional layer containing 24 filters with 3 × 3 kernels and ReLU activation. A 2 × 2 max-pooling layer was then used to reduce the spatial dimensions of the resulting feature maps while retaining prominent features. The extracted features were flattened and passed through two fully connected layers containing 128 neurons each, followed by a sigmoid output neuron for binary classification.

The CNN was trained using the Adam optimizer with a learning rate of 1x10^-6, binary cross-entropy loss, a batch size of 32, and five epochs, consistent with the assignment configuration. Training data were shuffled, and the same training and validation samples used for the baseline model were retained so that differences in performance could be attributed more directly to the change in model architecture. In addition to validation accuracy and MSE, the Pearson correlation coefficient was calculated between the model's predicted cat probabilities and the actual validation labels.

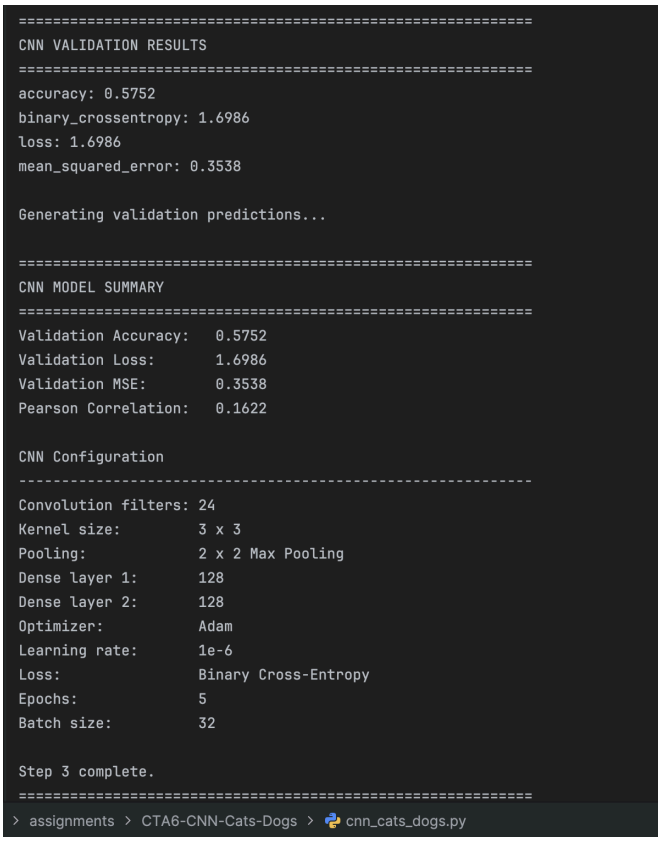
The single-convolution-layer CNN demonstrated an improvement over the fully connected baseline. In the initial evaluation, the model achieved 57.52% validation accuracy, compared with 50.00% for the baseline model, with an MSE of 0.3538 and a Pearson correlation of r = .1622. The positive correlation indicates that higher predicted cat probabilities were associated with actual cat labels, although the relatively small coefficient demonstrates that this relationship remained weak. Figure 4 presents the validation performance of the initial CNN.

The improvement illustrates the primary advantage of convolution for image analysis. Rather than requiring every pixel to connect independently to a dense layer, convolutional filters examine small local regions of an image and learn patterns that may be useful across different spatial locations. Max pooling further reduces the representation while retaining prominent learned features. As a result, the network can begin identifying visual characteristics that distinguish the two classes rather than relying solely on individual pixel values.

However, the improvement should not be interpreted as evidence that the CNN had become a strong classifier. Validation accuracy remained below 60%, and the weak Pearson correlation suggested considerable overlap between the predicted probabilities for cats and dogs. Additionally, repeated training demonstrated some variation in model performance, indicating that results from a single training run should be interpreted cautiously. The CNN therefore provided evidence that convolution was more appropriate than the fully connected baseline for this image-classification problem, but its performance also demonstrated that selecting an architecture designed for the data does not by itself guarantee reliable predictions.

Figure 4

Validation Performance of the Single-Convolution-Layer CNN



Note. The convolutional neural network used 24 3 × 3 filters, 2 × 2 max pooling, two 128-unit fully connected layers, the Adam optimizer with a learning rate of 1×10^{-6}, and binary cross-entropy loss. After five epochs, the model achieved 57.52% validation accuracy, a mean squared error of 0.3538, and a Pearson correlation of $r = .1622$ between predicted probabilities and validation labels. The CNN exceeded the 50.00% accuracy of the fully connected baseline, suggesting that preservation and extraction of spatial image features improved classification, although overall predictive performance remained limited.

Modification of the CNN Architecture

The CNN was subsequently modified to evaluate whether increasing the depth of the network would improve its ability to distinguish cats from dogs. The original convolutional layer containing 24 filters was retained, and a second convolutional layer containing 48 filters with 3 × 3 kernels and ReLU activation was added. A second 2 × 2 max-pooling operation was also applied following the new convolutional layer. The remainder of the architecture was kept consistent with the original CNN, including two fully connected layers containing 128 neurons each and a sigmoid output layer. The optimizer, learning rate, batch size, training duration, and validation dataset were also maintained to provide a more direct comparison between the two architectures.

The addition of a second convolutional layer allows the network to construct more complex representations from features identified by the preceding layer. The first convolutional layer operates directly on image pixels and can identify relatively simple local patterns, while subsequent convolutional layers operate on the resulting feature maps. In principle, this hierarchical structure allows deeper layers to combine lower-level visual features into increasingly complex representations. The use of 48 filters in the second layer also provides additional capacity for learning different feature patterns.

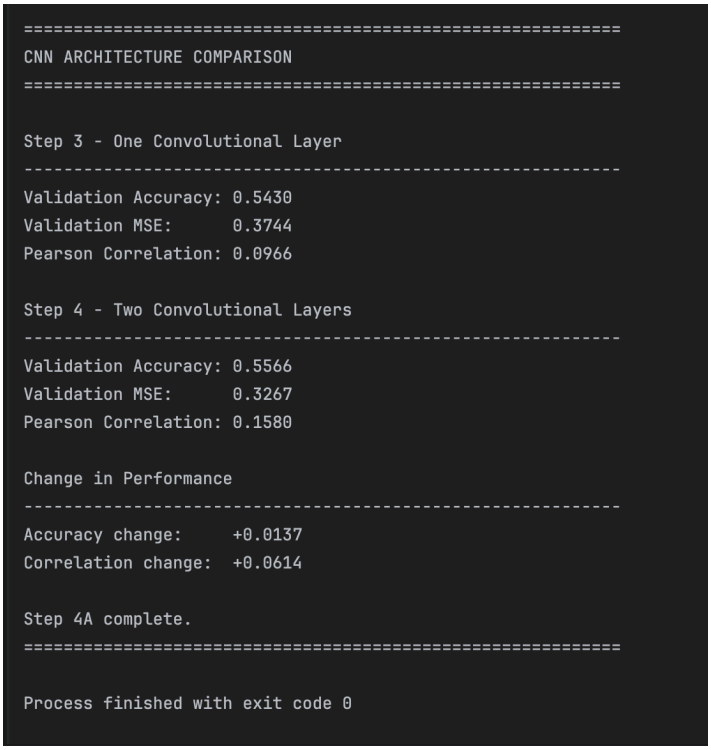
Within the comparative execution, the single-layer CNN achieved 54.30% validation accuracy, an MSE of 0.3744, and a Pearson correlation of $r = .0966$. After adding the second convolutional layer, validation accuracy increased to 55.66%, MSE decreased to 0.3267, and the Pearson correlation increased to $r = .1580$. Therefore, the new correlation coefficient requested in the assignment was $r = .1580$, representing an increase of .0614 relative to the single-layer model from the same execution. Validation accuracy increased by approximately 1.37 percentage points.

Figure 5 compares the performance of the one- and two-convolution-layer architectures. The results indicate that adding the second convolutional layer produced measurable improvements across accuracy, MSE, and Pearson correlation. However, the magnitude of the improvement was relatively small, and overall classification performance remained weak. The two-layer model continued to classify only slightly more than half of the validation images correctly.

An additional observation emerged when these results were compared with the earlier execution of the single-layer CNN, which had achieved 57.52% validation accuracy and $r = .1622$. The same architecture produced 54.30% accuracy and $r = .0966$ when retrained during the direct architectural comparison. This variation demonstrates that neural-network performance can differ across training runs because of factors such as parameter initialization and stochastic optimization. For this reason, the one- and two-layer results generated within the same execution provide the more appropriate basis for assessing the architectural modification.

Overall, adding another convolutional layer improved the model within the controlled comparison, but the results do not support the assumption that additional network complexity necessarily produces a proportionate improvement in predictive performance. Instead, the experiment reinforces the importance of evaluating architectural changes empirically rather than assuming that a deeper model is inherently superior.

Figure 5
Performance Comparison of One- and Two-Layer Convolutional Neural Networks



Note. Adding a second convolutional layer with 48 filters improved validation accuracy from 54.30% to 55.66% and reduced mean squared error from 0.3744 to 0.3267. The Pearson correlation between predicted probabilities and actual validation labels increased from $r = .0966$ for the single-convolution-layer model to $r = .1580$ for the two-layer model. Although the additional convolutional layer improved all three performance measures within this run, the relatively low validation accuracy and correlation indicate that increasing network depth alone did not produce strong classification performance.

Model Performance and Prediction Analysis

The final two-layer CNN was further evaluated by examining the distribution of its predicted probabilities rather than relying solely on classification accuracy. This analysis is important because a binary classifier using sigmoid activation produces a continuous probability between 0 and 1 before that value is converted into a categorical prediction. Using the standard classification threshold of .50, predictions greater than or equal to .50 were classified as cats, while predictions below .50 were classified as dogs. During this evaluation, the model achieved an overall validation classification accuracy of 57.03%. Although this exceeded the 50% expected from random classification on the balanced validation dataset, the result indicates that the model continued to misclassify a substantial proportion of images.

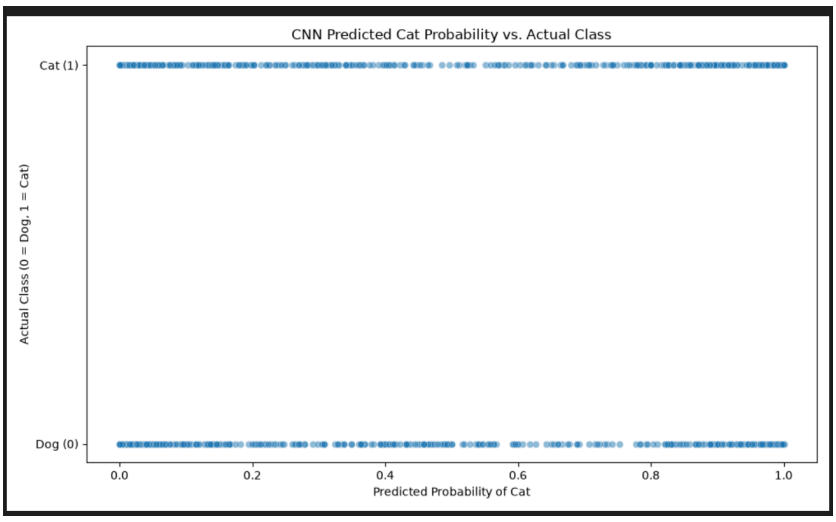
Figure 6 compares the CNN's predicted probability of an image being a cat with its actual classification. Because the true labels are binary, the observations form two horizontal groups representing dogs (0) and cats (1). A strong classifier would be expected to concentrate dog images near predicted probabilities of 0 and cat images near probabilities of 1. Instead, predictions for both classes were distributed across nearly the entire probability range. Some actual dogs received predicted cat probabilities approaching 1.0, while some actual cats received probabilities approaching 0.0. This overlap demonstrates that the CNN had not learned a sufficiently discriminative representation to consistently separate the two classes.

The probability-threshold analysis provided additional insight into this limitation. At a threshold greater than .10, 54.94% of selected images were actually cats. This proportion gradually increased as the threshold was raised, reaching 59.01% at a threshold greater than .70. However, the relationship did not continue as expected at higher confidence levels. Among the 265 images for which the CNN predicted a cat probability greater than .90, only 57.74% were actually cats. Figure 7 summarizes these threshold results.

This finding is particularly important when interpreting model output. A predicted probability of .90 might intuitively be interpreted as strong confidence that an image belongs to the predicted class. The results of this experiment demonstrate that such an interpretation would be misleading. Images receiving very high predicted cat probabilities were not substantially more likely to actually contain cats than images receiving more moderate probabilities. The CNN's numerical probability therefore represented the output of its learned decision function but did not provide a well-calibrated measure of prediction reliability.

The scatterplot, threshold analysis, and relatively weak Pearson correlation collectively demonstrate that the model learned some meaningful signal from the image data but remained unreliable as a classifier. This distinction highlights why accuracy alone is insufficient for evaluating artificial intelligence systems. Examining how predicted probabilities correspond with actual outcomes can reveal weaknesses that are obscured by a single aggregate performance metric. In applications where model outputs influence higher-stakes decisions, interpreting an uncalibrated confidence score as certainty could be considerably more problematic than an incorrect categorical prediction alone.

Figure 6
Predicted Cat Probabilities Compared With Actual Image Classifications



Note. The scatterplot compares the two-layer convolutional neural network's predicted probability that each validation image contains a cat with the image's actual class, where 0 represents a dog and 1 represents a cat. Predictions for both classes are distributed broadly across the probability range, indicating substantial overlap between cats and dogs. A stronger classifier would concentrate dog images toward probabilities near 0 and cat images toward probabilities near 1. The observed overlap is consistent with the model's relatively low validation accuracy and weak positive Pearson correlation between predictions and actual labels.

Figure 7

Classification Accuracy and Cat-Probability Threshold Analysis for the Two-Layer CNN

```
============================================================
STEP 4B - PREDICTION THRESHOLD ANALYSIS
============================================================

Standard Classification Results
------------------------------------------------------------
Classification threshold: 0.50
Classification accuracy:  0.5703
Classification accuracy:  57.03%

Cat Probability Threshold Analysis
------------------------------------------------------------
Threshold   Images      Actual Cats    Cat Proportion
0.1         708         389            0.5494
0.2         620         342            0.5516
0.3         566         313            0.5530
0.4         509         283            0.5560
0.5         458         265            0.5786
0.6         419         247            0.5895
0.7         383         226            0.5901
0.8         343         199            0.5802
0.9         265         153            0.5774

============================================================
STEP 4B SUMMARY
============================================================
Validation Accuracy:    0.5703
Pearson Correlation:    0.1498
Scatterplot saved as:  cnn_prediction_scatterplot.png
```

Note. At a classification threshold of .50, the convolutional neural network achieved 57.03% validation accuracy. The proportion of actual cats generally increased as the prediction threshold increased from .10 to .70, reaching a maximum of 59.01% at the .70 threshold. However, the proportion subsequently decreased to 57.74% among predictions greater than .90. These findings indicate that higher predicted probabilities did not consistently correspond with greater classification precision, suggesting that the model's probability estimates were poorly calibrated.

Individual Prediction Analysis

To examine the model beyond aggregate performance measures, the completed CNN was tested through an interactive interface that allowed individual validation images to be selected by index. For each image, the interface displayed the actual class, predicted class, probability of the image being a cat, corresponding probability of being a dog, and whether the classification was correct.

This provided a more intuitive way to evaluate the model's behavior and exposed an important limitation that was less apparent from accuracy alone.

Figure 8 demonstrates a successful classification using validation image 600. The image contained a dog, and the CNN correctly predicted the dog class. The model assigned only a 5.14% probability of the image being a cat, corresponding to a 94.86% probability of being a dog. This represents both a correct classification and a high-confidence prediction. Viewed independently, this example could create the impression that the CNN had learned a highly effective representation of the two classes.

Figure 10 provides a similar example for the opposite class. Validation image 325 contained a cat and was correctly classified as a cat. The CNN assigned an 82.15% probability to the cat class and a 17.85% probability to the dog class. Together, Figures 8 and 10 demonstrate that the network was capable of learning features that supported confident and correct classifications for examples from both classes.

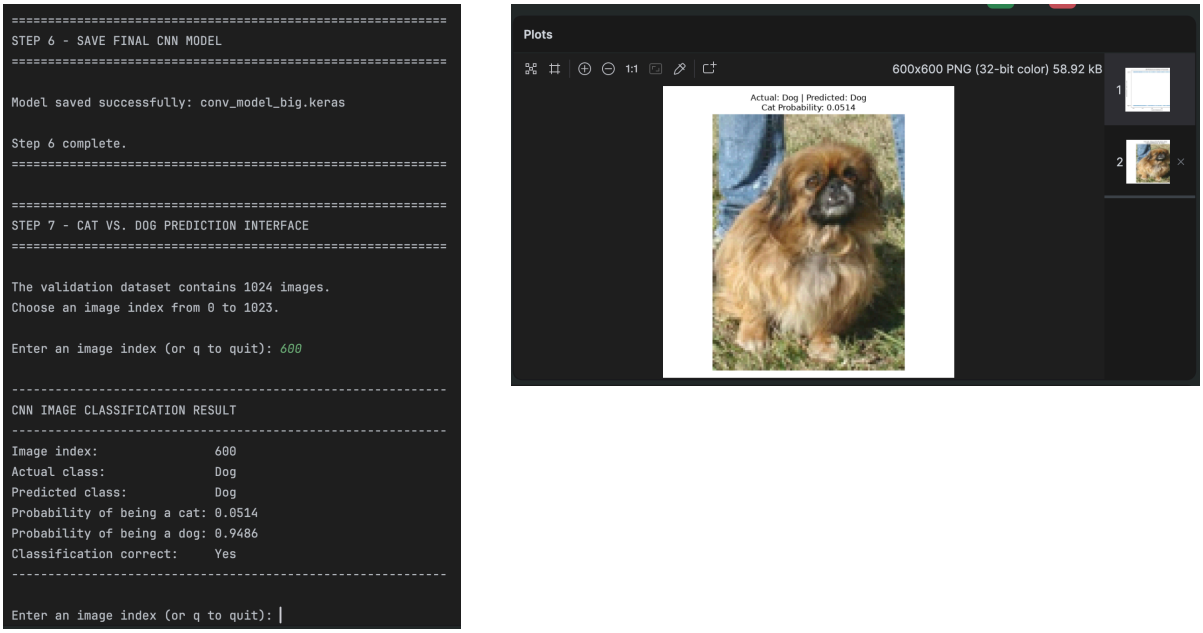
However, Figure 9 demonstrates why individual successful predictions should not be used as evidence of overall model reliability. Validation image 250 clearly contained a cat, yet the CNN classified the image as a dog. More importantly, this was not a marginal prediction near the .50 decision threshold. The model assigned only a 9.44% probability to the cat class and a 90.56% probability to the dog class. The network was therefore highly confident in an incorrect classification.

The comparison among these examples is particularly revealing. The model was 94.86% confident in its correct dog classification, 82.15% confident in its correct cat classification, and 90.56% confident in its incorrect dog classification of a cat. In other words, the model expressed greater confidence in one of its incorrect predictions than in one of its correct predictions. This observation reinforces the threshold-analysis results, which showed that increasing predicted probability did not consistently produce greater classification precision.

These examples illustrate an important distinction between model confidence and model veracity. A sigmoid output approaching either 0 or 1 indicates the strength of the model's output under its learned parameters; it does not independently establish that the prediction is trustworthy. A visually persuasive demonstration of one successful prediction can therefore provide a misleading impression of model quality if it is separated from validation accuracy, error analysis, calibration, and examples of model failure. Evaluating artificial intelligence requires examining not only where a model succeeds, but also how it fails and how confidently it can be wrong.

Figure 8

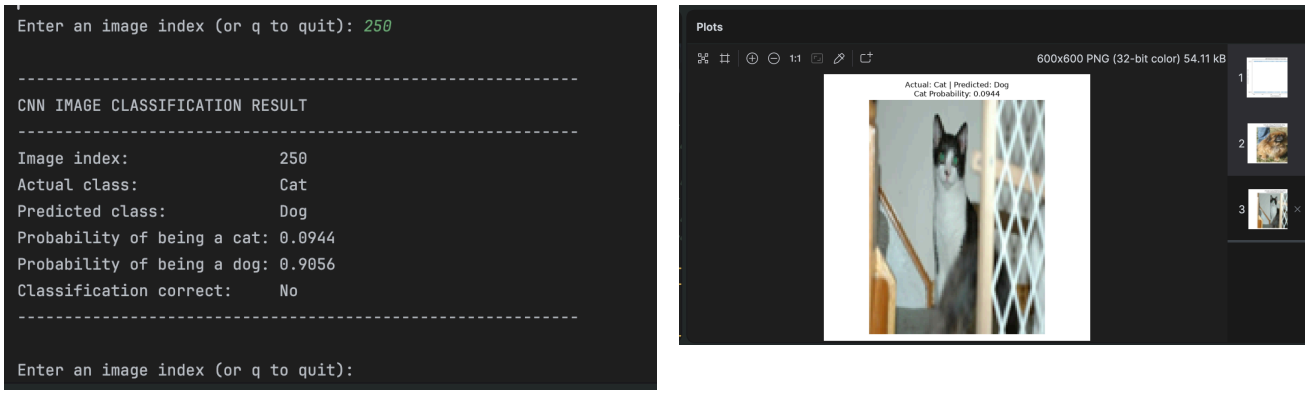
CNN Classification Interface Demonstrating a Correct Dog Prediction



Note. The user interface was tested using validation image 600. The two-layer convolutional neural network correctly classified the image as a dog, assigning a 5.14% probability of the image being a cat and, correspondingly, a 94.86% probability of it being a dog. This example demonstrates that despite relatively weak aggregate validation performance, the model can produce highly confident and correct classifications for individual images.

Figure 9

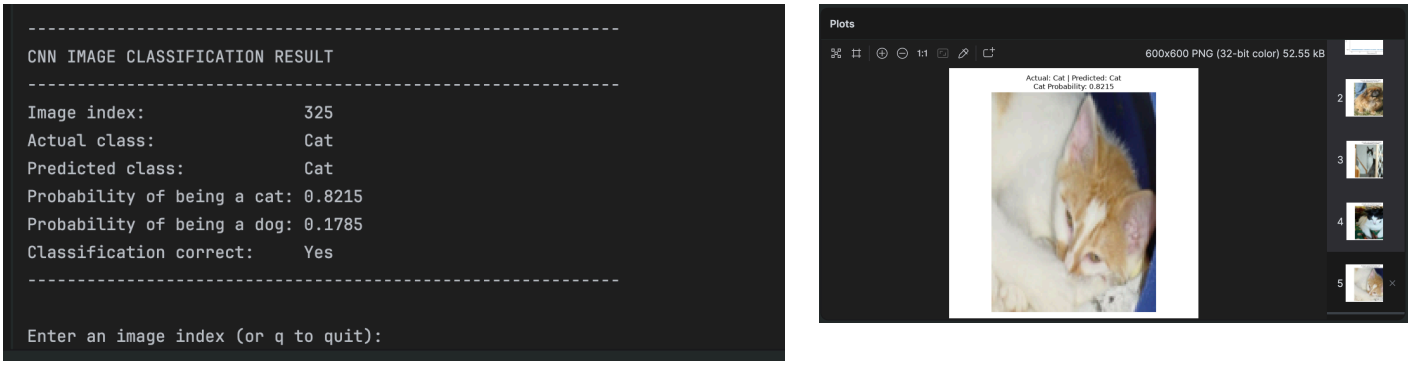
CNN Classification Interface Demonstrating a High-Confidence Misclassification



Note. Validation image 250 contains a cat but was incorrectly classified as a dog by the two-layer convolutional neural network. The model assigned only a 9.44% probability to the cat class and a 90.56% probability to the dog class. This high-confidence error demonstrates that predicted

probability should not necessarily be interpreted as model certainty or reliability. Although the network can make highly confident correct predictions, as demonstrated in Figure 8, it can also make highly confident incorrect predictions.

Figure 10
CNN Classification Interface Demonstrating a Correct Cat Prediction



Note. Validation image 325 contains a cat and was correctly classified by the two-layer convolutional neural network. The model assigned an 82.15% probability to the cat class and a 17.85% probability to the dog class. This example demonstrates that the CNN successfully learned image features capable of distinguishing some cat images from dog images, although aggregate validation results indicate that this performance was not consistent across the validation dataset.

Model Veracity, Limitations, and Opportunities for Improvement

The results of this experiment demonstrate that the final CNN possesses some predictive capability but should not be considered a reliable cats-versus-dogs classifier in its current form. The model consistently performed above the 50% chance level expected from the balanced validation dataset, and convolutional architectures generally performed better than the fully connected baseline. However, validation accuracy remained below 60%, Pearson correlations were weak, the two classes substantially overlapped in predicted probability, and individual testing demonstrated that the model could produce highly confident incorrect classifications. Collectively, these findings suggest that the CNN learned meaningful visual patterns without learning sufficiently robust features to generalize consistently to unseen images.

Several characteristics of the experimental design may have contributed to this performance. Most notably, the assignment retained raw RGB pixel values ranging from 0 to 255. Normalizing these values to a smaller range, such as 0 to 1, could provide more stable inputs during optimization and would be an appropriate first modification to test. The models were also trained for relatively few epochs, particularly the CNNs, which were limited to five epochs. Increasing

the training duration while monitoring validation performance with early stopping could determine whether the models were undertrained without simply allowing them to overfit the training data.

Data augmentation could also improve generalization. Transformations such as horizontal flipping, small rotations, cropping, zooming, and translation could expose the network to additional variations of the existing training images without requiring new labeled examples. This is particularly relevant to animal classification because cats and dogs can appear at different scales, orientations, backgrounds, lighting conditions, and positions within an image. Dropout or other regularization techniques could additionally be evaluated to reduce dependence on particular learned features.

The architecture itself could also be improved. Additional convolutional layers, batch normalization, alternative pooling strategies, and systematic hyperparameter tuning could potentially increase classification performance. However, the results of this experiment suggest that simply adding layers should not automatically be considered an improvement. Adding the second convolutional layer increased performance only modestly within the controlled comparison. A larger or deeper neural network introduces additional computational complexity without guaranteeing a meaningful increase in generalization.

A more practical approach may be transfer learning rather than continuing to build increasingly complex CNNs from scratch. Modern pretrained computer-vision networks have already learned general visual representations from substantially larger image datasets. Features learned by these networks can be adapted to a cats-versus-dogs classification task by replacing or fine-tuning their final classification layers. This could potentially provide stronger performance with less training data and computational effort than attempting to independently learn all relevant visual features from the 4,096-image training sample used in this experiment.

Finally, the model's probability outputs require additional attention. The threshold and individual-prediction analyses demonstrated that high probability values did not consistently correspond with high reliability. Future evaluation should therefore consider measures beyond accuracy, including precision, recall, F1 score, confusion matrices, receiver operating characteristic analysis, and probability-calibration techniques. Repeated training runs or cross-validation would also provide a more reliable estimate of performance than a single stochastic training run.

Overall, improving this model should not be approached as a search for the most complicated neural-network architecture. The appropriate goal is to identify the combination of preprocessing, architecture, training strategy, and evaluation methods that produces the most reliable generalization to unseen data. This project reinforces a recurring observation from previous experiments in this course: model sophistication and model quality are not synonymous. A more complex artificial intelligence model is valuable only when its additional complexity produces measurable and reproducible improvements in the problem it is intended to solve.

Conclusion

This project demonstrated the advantages and limitations of convolutional neural networks for image classification by comparing a traditional fully connected neural network with progressively more complex convolutional architectures. The fully connected baseline model achieved only 50% validation accuracy, demonstrating that simply increasing the number of trainable parameters is insufficient when the architecture does not appropriately represent the structure of the data. Introducing convolution and max pooling improved classification performance by allowing the model to learn spatial features within the images. Adding a second convolutional layer with 48 filters produced additional improvement, but the increase was modest and overall validation performance remained relatively weak.

The individual prediction and threshold analyses provided an equally important finding. The CNN was capable of making highly confident and correct predictions, but it was also capable of being highly confident and wrong. A probability approaching 1.0 or 0.0 therefore did not necessarily indicate that the prediction was reliable. This reinforces the importance of evaluating artificial intelligence using performance across unseen data rather than relying on individual demonstrations, model confidence, or architectural sophistication.

Perhaps the most important lesson from this experiment is that making a model more complicated does not necessarily make it better. The baseline network contained more than 18 million trainable parameters yet performed at chance level, while the addition of another convolutional layer produced only a modest improvement over the simpler CNN. Further increasing network depth may improve performance, but it may also add computational cost without addressing more fundamental limitations in preprocessing, training, or model selection. Techniques such as normalization, data augmentation, improved evaluation, or transfer learning may provide greater benefit than simply adding additional layers.

This conclusion extends beyond image classification. As increasingly complex artificial intelligence systems become available, there can be a tendency to equate sophistication with capability. The results of this project demonstrate why that assumption should be tested rather than accepted. The goal of machine learning is not to build the most complicated model; it is to build the simplest model that reliably solves the problem. Complexity is valuable when it produces measurable improvement. When it does not, more AI is not necessarily better AI.

References

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. https://www.deeplearningbook.org/

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning* (Vol. 70, pp. 1321–1330). PMLR. https://proceedings.mlr.press/v70/guo17a.html

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems, 25*. https://proceedings.neurips.cc/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html

Pan, S. J., & Yang, Q. (2010). A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering, 22*(10), 1345–1359. https://doi.org/10.1109/TKDE.2009.191

TensorFlow. (n.d.). *Convolutional neural network (CNN)*. https://www.tensorflow.org/tutorials/images/cnn